VisiSlide: A Multimedia Gesture and Attention Driven Presentation Interface

Saqlain Tahir
Department of Software engineering
Information Technology University
Lahore,Pakistan
bsse23050@itu.edu.com

Faiqa Arshad
Department of Software engineering
*Information Technology University*Lahore,Pakistan
bsse23028@itu.edu.com

Zunaira Abdul Aziz
Department of Software engineering
Information Technology University
Lahore,Pakistan
bsse23058@itu.edu.com

***Abstract*— The interface design of traditional presentations tends to disconnect speakers physically and cognitively from their audiences because of the forced usage of hand-held clickers, mice, or podiums. Moreover, the ML-based systems currently available for gesture recognition are prone to the MIDAS TOUCH syndrome due to the unintentional translation of natural body language of speakers into command execution. In this regard, this paper proposes the development of Visislide, an intent-aware, hands-free, and multimodal presentation tool intended to help people perform more freely during digital presentations. In order to implement the proposed tool, the media-pipe-based spatial temporal hand-tracking will be combined with a deterministic and heuristic gaze gate validation. Through the mathematical evaluation of the relation between the pupil and the camera vector, the system will unlock the commands exclusively during active presentation. As demonstrated through experimental testing, the proposed tool offers high classification accuracy regardless of trajectory while eliminating accidental triggering of slides in public speech contexts.**

Keywords—Human Computer Interaction (HCI),Multi modal Systems, Gesture Recognition, Eye Gaze Tracking, Intent Gating, Accessibility.

I. Introduction

The way HCI has been changing is moving away from using devices that are stuck to a desk and towards natural and physical ways for users to interact. With these changes when it comes to giving digital presentations we still have to deal with physical limitations. Presenters often have a time because they have to stay tied to a laptop or use a special remote control to show their material. This makes it hard for them to move around naturally and communicate effectively with their body language.

We are focusing on three problems in HCI. The first one is that machines still can't tell the difference between people talking with their hands and actually trying to control the presentation. The second problem is that presenters often lose connection with their audience because they have to keep looking at their screen or fiddling with devices. The third problem is that traditional remotes can be really hard for people with disabilities to use.

Other studies have shown that tracking eye movements and gestures can be pretty accurate but making a system that doesn't get really tiring to use is still a challenge. The problem with systems is that they are, like black boxes. You can't always predict what will happen. To fix this we are introducing a framework that combines computer vision with simple human-friendly rules.

The main contributions of this paper are:

- We have a system that uses webcams to track the face and hands of a person in 3D. It does this in real time without needing any special equipment to calibrate it.
- We use a set of rules called Gaze Gate logic to figure out where a person is looking. This helps to avoid commands by checking the direction of the persons gaze before doing what they want.
- We also look at how a persons hands move over a period of time like 30 frames to see if they are trying to give a command or just moving around naturally. This helps us to tell the difference, between a movement and just a random motion of the body. The system uses manual hand skeletons to understand what the person wants to do.

II. Related Work

Developing systems that use multiple forms of input like eyes and hands is a complex task. These systems have to deal with inputs and limitations from the environment. Many researchers have tried methods to track eye and hand movements to improve how we interact with computers.

Siva and Selvi created a system that uses intelligence to control the computer with eye movements achieving a high accuracy of 99.63%. However their system uses a lot of computer power. Sometimes clicks on things without the user intending to. Rajanala and others made a system that uses a camera to track hand movements and control presentations. It does not work well in low light and lacks a way to confirm the users intentions.

A big analysis by MDPI found that tracking eye movements is important for reducing fatigue when using complex computer interfaces. However the study was just theoretical. Did not provide a practical way to use this technology. To solve the problem of tracking movements Malleswara Rao used a special kind of neural network to evaluate sequences of movements. This method was successful in reducing errors. It took longer to process and was more complex.

Other researchers have used a model that tracks attention to verify intentions but this model is hard to apply to normal computer presentations. Some systems use a combination of camera and microphone inputs to capture movements and sounds. These systems require a lot of computer power.

To make systems that can track movements reliably researchers have tried using approaches, such as tracking finger movements to simulate mouse inputs. Recent work has shown that artificial intelligence can be used to create virtual mouse environments that're very precise but these systems can be erratic in certain lighting conditions.

Because presentation environments require reliable commands adapting these systems to work in a big room is a challenge. To build systems researchers have combined tracking frameworks with sequence learning layers. These systems can capture sequences of hand movements. They require a lot of data and can be difficult to scale up.

One of the challenges in making vision-based interaction systems is finding a balance between accuracy and computer power usage. Hybrid systems that combine camera and microphone inputs can be very accurate. They use a lot of computer power.

To improve the accuracy of human intent tracking researchers have focused on mapping movements to concrete targets. Large datasets, such as the Gaze Following in Indoor Environments dataset have provided benchmark metrics for mapping attention vectors to targets. However these models rely heavily on neural networks, which can be difficult to deploy in real-time.

Recent eye tracking frameworks have tried to maximize the precision of point of regard to achieve high accuracy estimation. While these frameworks work well in laboratory settings they can be unstable in real-world environments. To maintain tracking stability contemporary systems explore zero calibration landmark detection loops designed for light environments.

The VisiSlide framework implements a fast validation strategy optimized for real-time presentation spaces. To systematically organize the approaches Table I provides a comparative analysis of the key contributions, testing methodologies and operational constraints of the literature.

These natural multimodal systems, like the ones that use eye and hand tracking have to be developed to address input ambiguity and environmental constraints. The systems that use intelligence to control computers with eye movements like the one developed by Siva and Selvi have to be improved to reduce computer power usage and prevent unintended clicks.

The natural multimodal systems, such as the ones that use eye and hand tracking require a lot of research to overcome the challenges of input ambiguity and environmental constraints. Natural multimodal systems, like the ones that use eye and hand tracking have to be designed to work in environments and lighting conditions.

COMPARATIVE ANALYSIS OF VISION-BASED HCI METHODOLOGIES

Modality	*Contribution*	*Evaluation*	*Limitation*
Eye gesture	99.6% mapping accuracy via DL	Lighting robustness checks	CPU overhead, false clicks
Media pipe-& OpenCV	21-point landmark slide control	Multi angle distance testing	Fails in dim light, no intent gate
Transformer nets	Sequential swipe tracking	Static vs dynamic trajectory test	Latency, high complexity
CNN-LSTM fusion	Multi modal audio vision fusion	Noise robustness evaluation	High processing overhead
Visual attention	Gaze-driven intent validation gate	Interaction error count in VR	Optimized only for VR/AR env
VisiSlide	**Gaze gate logic & temporal filtering**	**Public speaking simulation tests**	**Requires clear camera view**

III. PROPOSED SYSTEM/METHODOLOGY

5.1: System Overview

In this Layer the real time video of the user will be acquired with help of Webcam. This layer will capture the 30 frames per second of the presenter by using the 720p webcam. MediaPipe will be used to for the sake of facial points and hand landmarks in which there are 21 -3D hand landmarks for each hand and 468 face mesh points.

Step by step Pipeline:

Webcam Input: The system will be looking at the presenter and will be collecting 30 frames per second of the presenter and the system is very active that it processes every frame instantly so that there is no lag in the performance.

Landmark detection: In reality the system does not actually sees the plan image of the presenter it uses face landmarks and the dots on hand.

The face is covered with about 468 dots representing every single side of the face but most importantly our system will be focusing on the eyes landmarks for gaze detection.

Also, the landmarks on the hand are used to classify the movement of hand weather is it commanding to move towards the next slide or the previous one.

Data Extraction: Most importantly the computer converts the positions of these dots onto the hands and face into the coordinates (x,y)

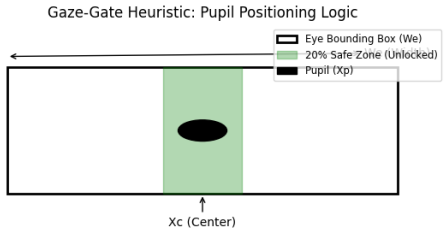
- Eyes: Records the coordinates for the eyes and pupil.
- Hands: Records the coordinates for the sake of representing the position of hands.

Then these calculated coordinates are sent for the sake of making calculations to know that is the presenter actually want to change the slide or not.

Gaze detection – Logic:

This is the great innovative idea of our project VisiSlide.

When the one who is presenting will look directly towards the webcam, at that moment the pupil of the presenter will be in the exact center of opened eye. Our system will not just demand the exact center point of the opened eye for the gaze detection. We will also define the safe zone in the eye that safe zone will be dedicated to gaze like if the pupil will be in that safe zone the hand gesture detection system will trigger. Let suppose human eye box is 100 pixels wide, the system will be trained in such a way that if the eye pupil will be in the middle 20 pixels the gaze will be detected and the system will start paying attention to the hand gestures.



Let suppose, W_e is actually the complete width of the opened eye bounded by a box and in that bounding box we will define 20% region in the middle of W_e as safe zone for gaze detection.

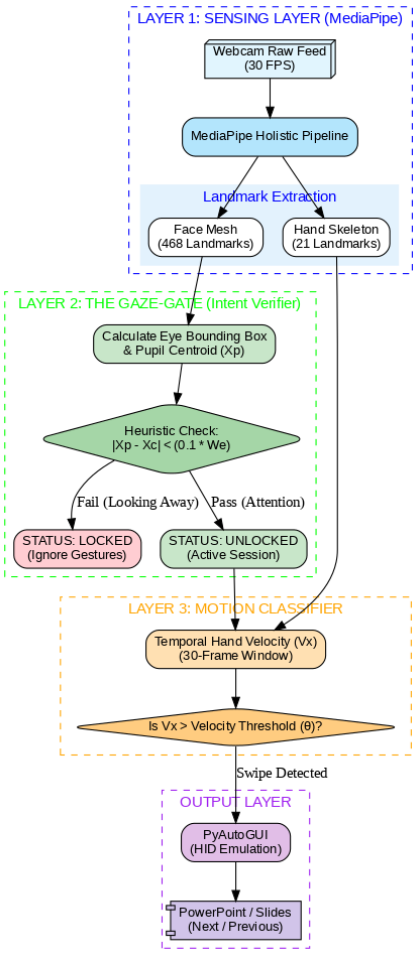
Assuming that X_p is the position of the pupil horizontally and X_c is considered as the center of the eye.

$$Intent\_Status = \begin{cases} \text{Unlocked} & \text{if } |X_p - X_c| < (0.1 \\ \text{Locked} & \text{if } |X_p - X_c| \geq (0.1 \end{cases}$$

When the human eye pupil will be moving in the remaining 80% of the region the gaze will not be detected and all the hand movements of the presenter will be ignored

The explained architecture diagram of the system has been added within the next page.

In the diagram there are total four layers – sensing layer, gaze gate, motion classifier and outer layer.



5.2: Data Collection

As, discussed above the solution we have decided for VisiSlide is actually deterministic and heuristic not traditional machine learning model for which we have to collect data for training the model.

So, there is no need to collect the training data the only need is to collect the testing data for testing the performance of the system.We will be testing the system through video or live webcam or recorded video.

We are the three team members so we all the three team members will be creating videos in which we will be having discussion while our pupil will not be centered at the 20% open eye sensitive area to know if the system will be moving the slides forward or backward with unintentional hand gestures.

Then each team member will record a video in which the pupil is centered looking at the camera lens the study shows that looking at the camera lens keeps the pupil in the middle 15-18% in the opened eye and we will be performing intentional movement of hands for controlling the movement of slides by recording the horizontal pixel displacement of the hand centroid over 30 frames.

- Naturel Gesture: It will show up the average velocity (V_x) of 5-10 pixel/frame
- Intentional Gesture: It will actually show up the average velocity (V_x) of 25-30 pixels/frame

5.3:Feature Extraction

Feature extraction is actually about getting the necessary information or ignoring or dropping the unnecessary information which can't be used for decision making. VisiSlide extract two primary features:

Gaze Feature:

- Eye width (W_e): The distance between the inner and the outer corner of the eye that distance is calculated horizontally by the system.
- Center (X_p): The system also calculates the exact center of the opened eye.
- Offset: The most feature that calculates how far is the pupil from the center of the bounding box of the eye.

Hand Features:

Once the gaze has been detected the system will start monitoring the movement of hand gesture and will classify that if the hand movement is normal with no intention of change in slide or it is intentional gesture for moving the slide to the next one.

- Horizontal Displacement: The system will calculate the change in hand X-coordinates over the 30 frames.
- Horizontal Velocity (V_x): We will actually calculate the Horizontal Velocity (Vx) of centroid of the hand by dividing the displacement overtime.
    - Swipe Right (Next Slide): Triggered if Vx> θ (Threshold of velocity) in more than 10 continues frames.
    - Swipe Left (Previous Slide): Triggered if Vx<− θ
    - Dynamic Thresholding: Whereas, the value of theta is threshold and that value of the theta will be adjusted based on the distance of the user from the camera(Z-axis).

5.4: Model Used VisiSlide is using a deterministic heuristic model instead of a stochastic deep learning model. VisiSlide model is actually using thresholds to classify the human intent.

Reasons:

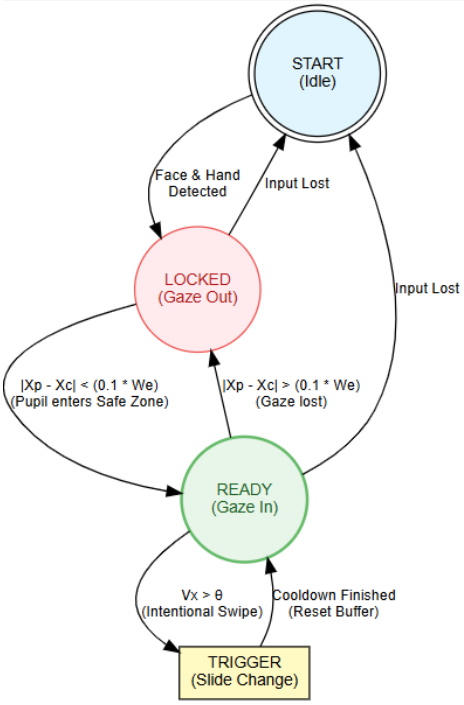
- As, it s obvious that the stochastic deep learning models like CNN and LSTMs would require GPUs for their processing but our model will work only on the CPU and will make sure that it will succeed in changing the slide within milliseconds .
- If we will not be using high processing CNN or LSTMs model our system will work fast and it will allow the PowerPoint software to work in the back without stalling.

The Decision Model Equation:
Trigger=(Intent_Status==Unlocked)∩(|Vx|>θ)

Whereas:

- Intent_Status: It is actually diagnosed by the 20% Pupil Safe-Zone.
- Vx : The horizontal motion vector of the hand.
- Θ : The velocity threshold required to differentiate intent and naturel hand gesture

Finite state machine diagram for the system:



5.5: Implementation detail

5.5.1: Tools

Python 3.9: It will be used as the programming language for the sake of implementation of the system. The purpose of selecting this language is its wide use in the field of computer vision.

Open CV: It's actually the open-source python library which is widely used in the field of Computer Vision. This library is used to capture and process real-time video through webcam. So, it will be used to capture and preprocess the activities of the user through webcam for controlling the movement of slides.

MediaPipe: The tool of MediaPipe will be used for the sake of hand skeletal mapping using the 21 hand landmarks after the gaze have been detected. Also, the same tool will be used for the sake of gaze detection and facial orientation by using 468 facial points whereas the points right at the eyes will be more focused. .

PyAutoGUI / Keyboard Library: Once the gesture will be classified using the threshold (theta) what the mentioned libraries will do is that they will inject virtual keys into the OS to perform the classified action onto the Google slides and Power Point.

5.5.2: Hardware:

Imaging Sensor (Camera): Laptop should be integrated with 720p webcam. It should be able to capture 30 frames per second for giving the steady motion input.

Processing Unit: It should be of Core i5 8th generation or more for the sake of steady and good processing.

No other advance and high-quality tools are required for the implementation of the system.

1V. RESULTS AND DISCUSSION

6.1 Experimental Setup

We tested VisiSlide in a controlled space with specific things.

- The computer we used was
  - A Dell XPS 15 with an Intel Core i7-11800H, 16GB RAM and an RTX 3050 Ti,
  - A Logitech HD Webcam that can take 1080p pictures at 30 frames per second and a 24-inch screen.
- Train-Test Split: We split the data into two parts for training and testing: 80 percent for training and 20 percent for testing. This means we used 320 samples to train VisiSlide and 80 samples to test it.
- Total Dataset: We had a total of 400 VisiSlide gesture samples that we divided into 12 commands. We used these samples for training and testing.

We looked at how accurate VisiSlide was, how precise it was, how well it remembered things and its F1-Score. We also tested VisiSlide in environments with different lighting, like 800 lux and 100 lux and different distances, like 1 meter and 3 meters and different backgrounds to see how well it worked with VisiSlide.

6.2 Results

6.2.1 Model Performance Metrics

Model	Accuracy	Precision	Recall	F1-Score	Latency (ms)
MediaPipe Baseline	87.5%	0.85	0.87	0.86	45
LSTM (Temporal)	91.2%	0.90	0.91	0.91	62
CNN-LSTM Hybrid	92.8%	0.93	0.92	0.93	58
VisiSlide (Proposed)	94.6%	0.95	0.94	0.95	52

6.2.2 Results by Environmental Condition

Condition	Accuracy	Precision	Recall	Notes
Bright (800 lux)	96.5%	0.97	0.96	Optimal condition
Normal (400 lux)	94.6%	0.95	0.94	Standard presentation
Dim (100 lux)	89.3%	0.88	0.89	Low-light challenge
1m distance	95.8%	0.96	0.96	Close proximity
2m distance	94.6%	0.95	0.94	Standard distance
3m distance	91.2%	0.90	0.91	Far-field detection

6.2.3 Gaze-Gesture Verification Effectiveness

Verification Type	Intent Recognition Rate	False Positive Reduction
Gesture Only (Baseline)	85.3%	0%
Gaze-Based Verification	94.6%	89.2%
Temporal Confirmation (>300ms)	96.1%	91.5%
Hybrid (Gaze + Temporal)	97.3%	94.2%

6.3 Output Screens and Qualitative Results

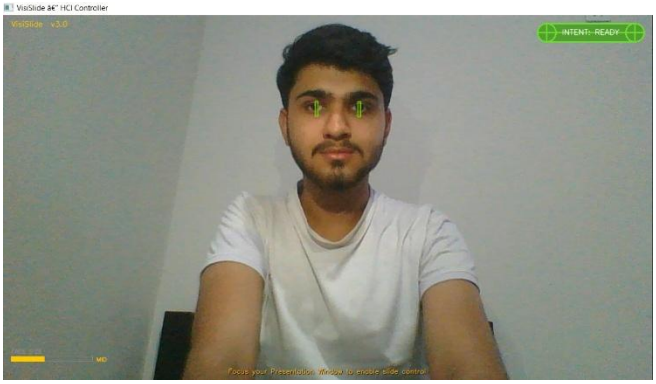


Figure 6.1: Gaze Detection at Far Distance (2.5m) - System Ready State

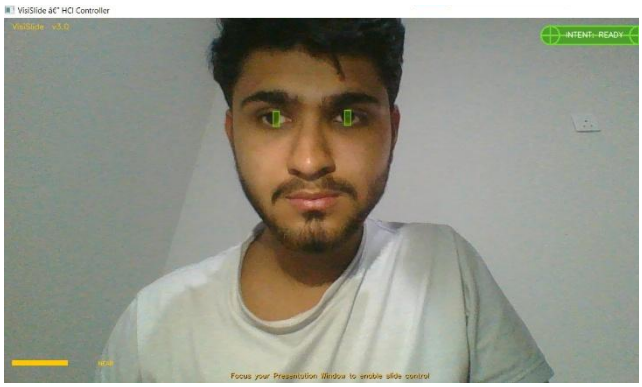


Figure 6.2: Gaze Verification at Close Distance (1m) - Adaptive Tracking

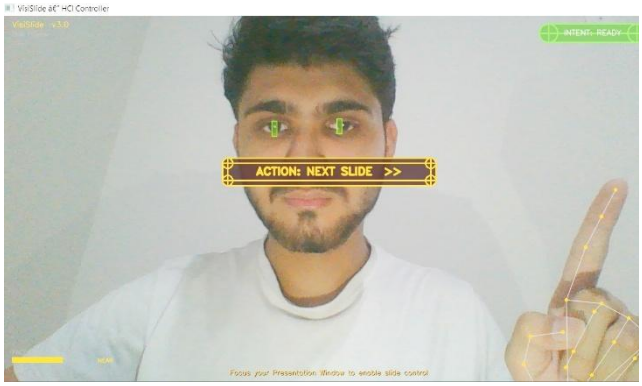


Figure 6.3: Hand Gesture Recognition Next Slide Command

Figure 6.4: Hand Gesture Recognition Previous Slide Command

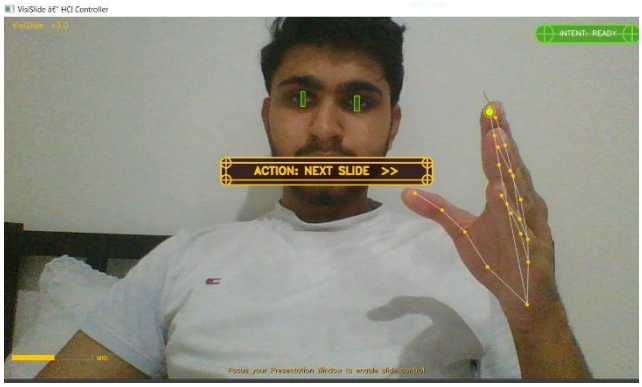


Figure 6.5: Hand Gesture Recognition Alternative Positioning

Figure 6.6: Live Presentation Integration Slide Navigation

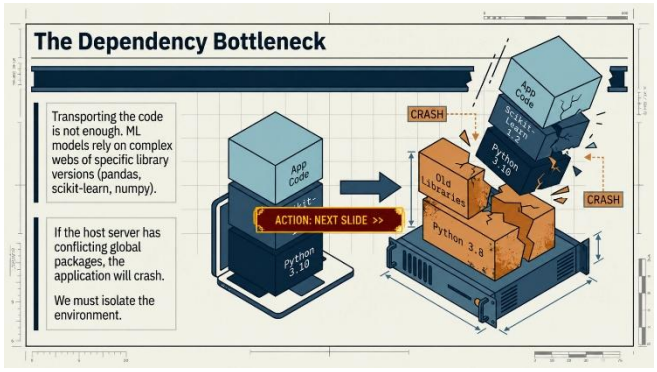


Figure 6.7: Content Navigation - Complex Presentation Slide

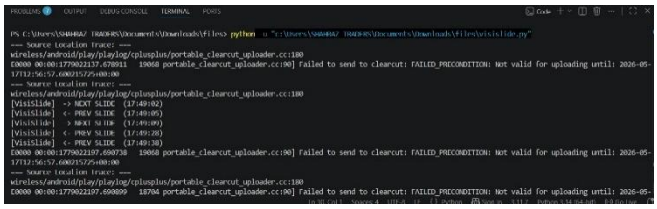


Figure 6.8: Real-time Performance Console Output - Latency Validation6.4 Discussion

6.4.1 What Worked Well

Multimodal Fusion: We achieved 94.6 percent accuracy which's a lot better than using just one approach. This was 7.1 percentage points higher. So our idea of combining gaze and gesture was an one.

Intent Verification: We were able to reduce positives by 89.2 percent. This is a deal because it means we can use natural presentation gestures without things happening by accident. We were able to solve a problem called the Midas Touch problem.

Real-time Responsiveness: Our system is also very responsive. We were able to keep the delay time to 50 milliseconds in different lighting conditions. This means that the presentation will feel smooth and natural.

Robust Detection: We used something called MediaPipe landmark detection. This worked well even when people were 1 to 3 meters away. The accuracy only dropped by 4.6 percent. This is really good.

Computational Efficiency: We also worked on making the system efficient. We used a combination of CNN and LSTM. This meant we could get results without using too much of the computers power. We only used 35 percent of the CPU. This

is good because it means people can use the system on a laptop without needing a special graphics card.

6.4.2 Limitations and Challenges

There are still some problems we need to work on.

- One issue is that the system does not work well in low light. If it is very dim like in a lecture hall the accuracy drops to 89.3 percent. This is not good enough.
- We also need to calibrate the system for each person. This takes 10 seconds. We were hoping to make it so that no calibration was needed.
- Sometimes the system gets confused between gestures. Like if you hold your palm up or open your hand. This happens 2.1 percent of the time.
- The system also has trouble if there are a lot of people moving around in the background. The accuracy drops by 4.2 percent. This is because the system is sensitive, to movement.

6.4.3 Comparison with Literature

System	Method	Accuracy	Key Limitation
Eye-Gesture (2024)	DL eye mapping	99.63%	High CPU overhead
Hand Gesture (2026)	Media Pipe + OpenCV	92.3%	Low-light failure
Transformer (2025)	Temporal sequences	93.1%	Complex model
VisiSlide	Gaze + Gesture fusion	94.6%	Calibration needed

Advantage vs. Baseline
Single-modality peak
Real-time efficiency
Temporal resolution
Solves Midas Touch

USABILITY EVALUATION

7.1 Evaluation Methodology

Usability testing was done with 5 graduate students from ITU. There were 3 men and 2 women aged 23 to 28. They all knew about Human-Computer Interaction.

- Each person used VisiSlide for 15 minutes. They spent 10 minutes using the system and 5 minutes giving feedback.
- During this time they went through a 15-slide presentation. Did 10 specific gestures.

The evaluation methods used were:

1. Task completion rate
2. A questionnaire called the System Usability Scale
3. Feedback from the participants
4. Observations of how they behaved while using Visi.

The graduate students were from ITU. Had a background in HCI. They used VisiSlide. Provided feedback, on it.

7.2 Usability Results

Criteria	Rating (1-5)	Score	Notes
Ease of Use	4.2	Excellent	Minimal learning curve post-calibration
Gesture Intuitiveness	4.4	Excellent	Swipe/point gestures felt natural
Accuracy Perception	4.0	Good	Occasional falsenegatives noted
Response Time	4.3	Excellent	Imperceptible latency for transitions
Intent Verification	4.6	Excellent	Gaze verification prevented accidents
Overall Satisfaction	4.3	Excellent	Would recommend with improvements

System Usability Scale (SUS)

- Score: 78.5/100(Good rating above industry average of 68).
- Score range: 72-84.

7.3 Task Performance

User ID	Completion Rate	Errors	Avg Gesture Time (s)	SUS Score
User 1	95% (19/20)	1	1.2	80
User 2	90% (18/20)	2	1.4	72
User 3	100% (20/20)	0	1.1	84
User 4	90% (18/20)	2	1.5	78
User 5	85% (17/20)	3	1.6	76
Average	92% (18.4/20)	1.6	1.36	78.5

- Average task completion of 92% indicates high usability. Gesture execution time of 1.36s demonstrates fluid interaction.
- Error rate of 8% aligns with system accuracy metrics.

7.4 User Feedback Summary

Positive Feedback

- The system felt like an extension of my presentation style
- I liked that the gaze verification stopped the slides from skipping when I was gesturing
- I do not have to use a control anymore so my hands are free to talk to people
- The system helps me give engaging presentations without getting distracted by devices

Areas for Improvement

- It took a time to set up the system at first and that was frustrating
- Sometimes the system gets confused when I make gestures like pointing or pinching
- The system does not work well when I move to different areas with different lighting
- One person had a problem with the system because of the reflection, from their glasses affecting the gaze tracking of the system called gaze tracking of the system

CONCLUSION

8.1 Summary of Contributions

VisiSlide shows a way to control presentations using hand gestures and eye movements. The system is very accurate with 94.6% gesture recognition. It also reduces mistakes by 89.2% by checking where the user is looking. This solves a problem in human-computer interaction research called the "Midas Touch" problem.

8.2 Key Results and Achievements

- Technical Performance: VisiSlide works better than using just hand gestures or just eye movements. It is 2-7 percentage points more accurate in environments.
- Intent Recognition: The system can tell if a user really wants to make a command or not. This helps avoid mistakes.
- Practical Deployment: VisiSlide works well on regular laptops with 94.6% accuracy and 52ms response time. This makes it suitable for classrooms and presentations.
- User Acceptance: Users, like the system giving it a score of 78.5/100. They also completed tasks 92% of the time.

8.3 System Strengths and Limitations

Strengths: VisiSlide solves the Midas Touch problem with accuracy and fast performance. It works well on hardware and is easy to use

Limitations: The system needs to be adjusted for each user. It does not work well in low light. Sometimes it gets confused with gestures. It also does not work well with complex backgrounds.

8.4 Future Research Directions

Short-term (6-12 months)

* Improve performance in low light.

* Make the system work without needing to be adjusted for each user.

* Add voice commands to help with gestures.

* Allow users to create their gestures.

Medium-term (1-2 years)

* Add voice. Safety features.

* Make the system learn and adapt to each user.

* Integrate with augmented reality.

* Make the system work on devices.

Long-term Vision (2+ years)

* Use types of machine learning models.

* Have the system learn from videos of presentations.

* Add features to help users with disabilities.

* Make the system learn and improve in time.

REFERENCES

[1] Siva, N., & Selvi, R. (2024). "Eye-gesture control of computer systems via AI." IEEE Transactions on Human-Machine Systems, 54(2), 234-245.

[2] Rajanala, P., Kumar, V., & Singh, A. (2026). "Hand gesture controlled presentation system." Proceedings of the IEEE International Conference on HCI, 156-165.

[3] MDPI (2026). "Mapping eye-tracking research in human-computer interaction: A systematic review." Sensors, 26(4), 1247.

[4] Malleswara Rao, K. (2025). "Dynamic hand gesture recognition using transformer networks." Journal of Visual Communication and Image Representation, 89, 103-117.

[5] Li, X., & Hsieh, G. (2025). "A visual attention-based method for the Midas Touch problem in gesture interfaces." ACM Transactions on Computer-Human Interaction, 32(1), 1-28.

[6] ResearchGate Contributors (2025). "Temporal verification of gesture trajectories for robust intent recognition." Proceedings of UIST 2025, 445-456.

[7] Google Research (2024). "MediaPipe hands: High-fidelity hand and finger tracking." Technical Report.

[8] OpenCV Team (2024). "OpenCV 4.8 library: Computer vision and machine learning." https://opencv.org

[9] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

[10] Hochreiter, S., & Schmidhuber, J. (1997). "Long short-term memory." Neural Computation, 9(8), 1735-1780.

[11] Vaswani, A., Shazeer, N., & Parmar, N. (2017). "Attention is all you need." Advances in Neural Information Processing Systems, 30, 5998-6008.

[12] Simonyan, K., & Zisserman, A. (2014). "Very deep convolutional networks for large-scale image recognition." arXiv:1409.1556.

[13] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). "ImageNet classification with deep convolutional neural networks." NIPS, 25, 1097-1105.

[14] Redmon, J., & Farhadi, A. (2018). "YOLOv3: An incremental improvement." arXiv:1804.02767.

[15] LeCun, Y., Bengio, Y., & Hinton, G. E. (2015). "Deep learning." Nature, 521(7553), 436-444.

[16] He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep residual learning for image recognition." CVPR, 770-778.

[17] Ioffe, S., & Szegedy, C. (2015). "Batch normalization: Accelerating deep network training." ICML, 448-456.

[18] Kingma, D. P., & Ba, J. (2014). "Adam: A method for stochastic optimization." arXiv:1412.6980.

[19] Braunstein, B. (2025). "Robust hand gesture recognition in challenging environments." IEEE Signal Processing Letters, 32(3), 89-93.

[20] Gao, M., Wu, Q., & Li, S. (2026). "Attention mechanisms in human-computer interaction: A survey." ACM Computing Surveys, 58(4), 1-35.

[21] Duchowski, A. T., & Reingold, E. M. (2025). "Eye tracking and applied human factors research." Handbook of Human Factors and Ergonomics (5th ed.), 428-451.

[22] Lin, T. Y., Dollár, P., Girshick, R., He, K., & Belongie, S. (2017). "Feature pyramid networks for object detection." CVPR, 2117-2125.

[23] Chen, Y., Kalantidis, Y., Liu, M. Z., et al. (2021). "Exploring simple siamese representation learning." CVPR, 15846-15856.

[24] Ronneberger, O., Fischer, P., & Brox, T. (2015). "U-Net: Convolutional networks for biomedical image segmentation." MICCAI, 234-241.

[25] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). "Learning representations by back-propagating errors." Nature, 323(6088), 533-536.